

Adatbázis kezelés II.

Procedure

Rostagni Csaba

2025. március 11.

Ezen az órán... I

1 Bevezető

2 Eljárások

Tartalom I

1 Bevezető

Tárolt programok - Stored Programs

Ismétlődő feladatokat, lekérdezéseket egyszerűsíthatunk úgy, hogy egy tárolt eljárást készítünk, majd megfelelően felparaméterezve futtatjuk. Ezzel programozhatunk SQL-ben.

- Az adatbázis szerveren futtatható programkód.
- Típusai
 - Eljárások (Stored procedures)
 - Függvények (Stored functions)
 - Triggeresk (Triggers)
- Biztonságosabba teheti az adatbázist
- Hálózati forgalom csökkenhető (már feldolgozott adatokat küldünk, fölöslegeseket nem)
- Hatékonyabb lehet az adatbázis művelet
- Elég egyszer lefejleszteni, bárholnan használható

Persistent Stored Modules - SQL/PSM

- SQL szabvány a tárolt programokra.
- Megvalósításai:
 - MySQL, MariaDB
 - Oracle (PL/SQL)
 - Microsoft, Sybase (T-SQL)
 - ...

<https://en.wikipedia.org/wiki/SQL/PSM>

Tartalom I

2 Eljárások

- phpMyAdmin
- Unbounded SELECT
- SELECT INTO
- IF

Tárolt eljárások dokumentációban

```
CREATE
    [DEFINER = user]
PROCEDURE sp_name ([proc_parameter[,...]])
    [characteristic ...] routine_body
```

proc_parameter: param_name type

func_parameter:
 param_name type

type: Any valid MySQL data type

characteristic:
 COMMENT 'string'
 | LANGUAGE SQL
 | [NOT] DETERMINISTIC
 | { CONTAINS SQL | NO SQL | READS SQL DATA | MODIFIES SQL DATA }
 | SQL SECURITY { DEFINER | INVOKER }

routine_body: Valid SQL routine statement

Tárolt eljárások

- A lekérdezés során az eredményt két féle képpen tárolhatjuk el.
 - SELECT INTO
 - CURSOR
- Ha nem tároljuk el, akkor "unbounded" lesz a lekérdezés

[https://dev.mysql.com/doc/connector-net/en/
connector-net-tutorials-stored-procedures.html](https://dev.mysql.com/doc/connector-net/en/connector-net-tutorials-stored-procedures.html)

Tárolt eljárás paraméterei

- Adat mozgás iránya szerint lehet

IN Bemenei

OUT Kimeneti

INOUT Ki- és bemeneti

```
IN nev VARCHAR(20)
```

Megadása a fenti minta szerint: Irány, majd a név végül a típus.

Tartalom

2 Eljárások

- phpMyAdmin
- Unbounded SELECT
- SELECT INTO
- IF

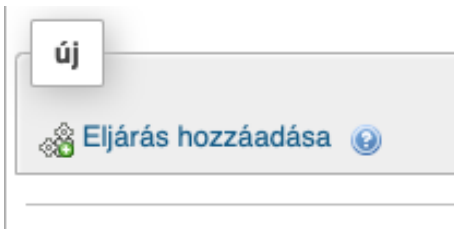
Tárolt eljárások phpMyAdmin-ban

- phpMyAdmin-ban tárolt eljárást az adatbázis kiválasztása után tehetjük meg.
- A tárolt eljárások az adatbázishoz kapcsolódnak
- Az "Eljárás hozzáadása" felírra kattintva hozhatunk létre újat.



Figyelem!

A tábla kiválasztása után nem ajánlja fel ezt az opciót! A gombot elrejtí.



Tárolt eljárások phpMyAdmin-ban

Eljárás hozzáadása

Részletek

Eljárás neve

Típus

PROCEDURE

Paraméterek

	Irány	Név	Típus	Hossz/ Értékek	Beállítások	
	IN		INT			Eldobás

Meghatározás

Paraméter hozzáadása

1

Determinisztikus

☐

Meghatározó

Biztonság típusa

DEFINER

SQL-adathozzáférés

NO SQL

Megjegyzés

Tárolt eljárások

- Az **Eljárás neve** [Name] (ezzel hivatkozhatunk rá később)
- A **Típus** [Type] részben adjuk meg, hogy [PROCEDURE] szeretnénk létrehozni.
- A paramétereknél [Parameters] kitöltendő:
 - Ki vagy bemeneti paraméterről van szó
 - A paraméter neve (ezzel hivatkozhatunk rá a tárolt eljáráson belül) A kukac itt nem szerepel, ezek másmilyen változók.
 - A típusa, illetve a típushoz tartozó további paraméterek
 - Több paramétert a **Paraméter hozzáadása** gombbal adhatunk hozzá, a fölöslegeseket pedig az **Eldobás** szüntethetjük meg.
- A **Meghatározás** [Definition] részben írhatjuk meg a kódunkat.

Tárolt eljárások

- A **Determinisztikus** kódunk azonos bemenet esetén azonos kimenetet produkál.
- A **Meghatározó** [Definer] segítségével adjuk meg, hogy melyik felhasználó hozta létre.
- A **Biztonság típusa** [Security type] adja meg, hogy a létrehozó vagy a futtató felhasználó nevében fusson-e az eljárás.
- Az **SQL-adathozzáférés** arról tárol el információkat, hogy a kódunk tartalmaz -e SQL-t.
 - READS SQL: Csak olvas az adatbázisból
 - MODIFIES DATA: Módosít az adatbázisban található adatokon.
 - NO SQL: Nem tartalmaz SQL-t.
 - CONTAINS SQL: Tartalmaz SQL-t.

Tartalom

2 Eljárások

- phpMyAdmin
- **Unbounded SELECT**
- SELECT INTO
- IF

Unbounded SELECT statements

- Inkább tesztelésre használatos!
- Az eljárás lefutása közben információt szolgáltatathatunk.
- Könnyű megérteni.

`https://dev.mysql.com/doc/connector-net/en/
connector-net-tutorials-stored-procedures.html`

Tárolt eljárások példa (unbounded)

Jelenítsük meg a Ford gyártmányú autókat.

MySQL

```
DELIMITER //
```

```
CREATE PROCEDURE `fordok`()  
  READS SQL DATA  
BEGIN  
  SELECT * FROM `autok`  
  WHERE `gyarto` = 'Ford';  
END//
```

```
DELIMITER ;
```

Tárolt eljárások példa (unbounded)

Jelenítsük meg a Ford gyártmányú autókat.

MySQL

```
CALL fordok();
```

ABC-123	Ford	Focus	M1	diesel	kék
AA-AX-1234	Ford	Fiesta	M1	benzin	sárga

Eredményként megkapjuk az eljárásban lefuttatott lekérdezés eredményét.

Tárolt eljárások példa (unbounded)

Jelenítsük meg a megadott gyártmányú autókat.

```
DELIMITER //  
  
CREATE PROCEDURE `gyartmany`(IN gyarto VARCHAR(20))  
  READS SQL DATA  
BEGIN  
  SELECT * FROM `autok`  
  WHERE `gyarto` = gyarto;  
END//  
  
DELIMITER ;
```

logikai hiba

Tárolt eljárások példa (unbounded)

Jelenítsük meg a megadott gyártmányú autókat.

MySQL

```
CALL gyartmany('Ford');
```

XXX-111	Opel	Adam	M1	benzin	piros
AAA-555	Honda	Jazz	M1	hibrid	kék
ABC-123	Ford	Focus	M1	diesel	kék
AA-AX-1234	Ford	Fiesta	M1	benzin	sárga

Figyelem!

Az eljárás paraméter neve megegyezik a tábla egyik mezőjével. Így minden sort visszaad. A mezőt hasonlítja össze az mezővel. Még a backtick sem segít.

Tárolt eljárások példa (unbounded)

Jelenítsük meg a megadott gyártmányú autókat.

MySQL

```
DELIMITER //
```

```
CREATE PROCEDURE `gyartmany` (IN gy VARCHAR(20) )  
    READS SQL DATA  
BEGIN  
    SELECT * FROM `autok`  
    WHERE `gyarto` = gy;  
END//
```

```
DELIMITER ;
```

Tárolt eljárások példa (unbounded)

Jelenítsük meg a megadott gyártmányú autókat.

MySQL

```
CALL gyartmany('ford');
```

ABC-123	Ford	Focus	M1	diesel	kék
AA-AX-1234	Ford	Fiesta	M1	benzin	sárga

Tartalom

2 Eljárások

- phpMyAdmin
- Unbounded SELECT
- **SELECT INTO**
- IF

SELECT INTO változónév

Amennyiben csak egy sort ad vissza a lekérdezésünk, az INTO segítségével betölthetjük változó(k)ba az eredményt.

- Egy sort kell a lekérdezésnek visszaadnia!
- Ugyanannyi mezőt kell lekérdezni, mint ahány mezőnk van.
- Allekérdezésekkel nem használható.
- UNIO esetén vannak bizonyos megkötések.
- Prepared statement esetén csak felhasználói változóba tud lekérdezni!

<https://dev.mysql.com/doc/refman/8.0/en/select-into.html>

SELECT INTO példa

Határozza meg eljárás segítségével, hogy adott gyártmányú autóból hány található meg az adatbázisban.

MySQL

```
DELIMITER //
```

```
CREATE PROCEDURE gyartmany_db(IN gy VARCHAR(20), OUT darab INT)  
  READS SQL DATA  
BEGIN  
  SELECT COUNT(*)  
    INTO darab  
  FROM `autok`  
  WHERE `gyarto` = gy;  
END//
```

```
DELIMITER ;
```

SELECT INTO példa

Határozza meg eljárás segítségével, hogy adott gyártmányú autóból hány található meg az adatbázisban.

MySQL

```
CALL gyartmany_db('Ford',@db);  
SELECT @db AS `Darab` FROM DUAL;
```

Darab
2

- Az eredményt a metódus a kimeneti változón keresztül adja meg.
- Az eredményt a `@db` felhasználó változóban tároljuk ideiglenesen.
- SELECT segítségével jelenítjük meg a `@db` eredményét.

SELECT INTO példa

Eljárás segítségével állapítsa meg rendszám alapján a jármű gyártóját és típusát.

MySQL

```
DELIMITER //  
CREATE PROCEDURE info(  
    IN _rendszam VARCHAR(7),  
    OUT _gyarto VARCHAR(20),  
    OUT _tipus VARCHAR(20)  
)  
    READS SQL DATA  
BEGIN  
    SELECT `gyarto`, `tipus`  
    INTO _gyarto, _tipus  
    FROM `autok`  
    WHERE `rendszam` = _rendszam;  
END//  
DELIMITER ;
```

Egy változó neve kezdődhet akár aláhúzással, így biztosan nem keveredik össze a mezők nevével.

SELECT INTO példa

Eljárás segítségével állapítsa meg rendszám alapján a jármű gyártóját és típusát.

MySQL

```
SET @rendszam='ABC-123';  
CALL `info`(@rendszam, @gyarto, @tipus);  
SELECT @gyarto AS `Gyártó`, @tipus AS `Típus` FROM DUAL;
```

Gyártó	Típus
Ford	Focus

- A példában a be és a kimeneti változók is.

Tartalom

2 Eljárások

- phpMyAdmin
- Unbounded SELECT
- SELECT INTO
- IF

Elágazások - IF statement

```
IF search_condition THEN statement_list  
    [ELSEIF search_condition THEN statement_list] ...  
    [ELSE statement_list]  
END IF
```

- Semelyik `statement_list` nem lehet üres, legalább 1 SQLt tartalmazzon!

<https://dev.mysql.com/doc/refman/8.0/en/if.html>

Elágazás példa

MySQL

```
DELIMITER //
```

```
CREATE PROCEDURE alkohol(IN kor INT)
```

```
BEGIN
```

```
    if kor < 18 THEN
```

```
        SELECT 'Nem ihat alkoholt' AS `Válasz` FROM DUAL;
```

```
    ELSE
```

```
        SELECT 'Ihat alkoholt' AS `Válasz` FROM DUAL;
```

```
    end if;
```

```
END//
```

Elágazás példa

MySQL

```
CALL alkohol(20);
```

Válasz

Ihat alkoholt

MySQL

```
CALL alkohol(17);
```

Válasz

Nem ihat alkoholt

Elágazás példa

MySQL

```
DELIMITER //
```

```
CREATE PROCEDURE alkohol2(IN kor INT)  
BEGIN  
    if kor < 18 THEN  
        SELECT 'Nem ihat alkoholt' AS `Válasz` FROM DUAL;  
    ELSEIF kor <21 THEN  
        SELECT 'Nem mindenhol ihat alkoholt' AS `Válasz` FROM DUAL;  
    ELSE  
        SELECT 'Bárhol ihat alkoholt' AS `Válasz` FROM DUAL;  
    end if;  
END;  
  
END//
```

Elágazás példa

MySQL

```
CALL alkohol(20);
```

Válasz
Nem mindenhol ihat alkoholt

MySQL

```
CALL alkohol(17);
```

Válasz
Nem ihat alkoholt

Tárolt eljárások példa: Ékezetlenítés

MySQL

```
DELIMITER //
```

```
CREATE PROCEDURE sp_ekezetlenit(INOUT nev VARCHAR(20))  
BEGIN
```

```
    SET nev = REPLACE(nev, 'á', 'a');
```

```
    SET nev = REPLACE(nev, 'Á', 'A');
```

```
    SET nev = REPLACE(nev, 'é', 'e');
```

```
    SET nev = REPLACE(nev, 'É', 'E');
```

```
    ...
```

```
END//
```

```
DELIMITER ;
```

Tárolt eljárások példa: Ékezetlenítés

MySQL

```
SET @nev = 'Áron';  
CALL sp_ekezetlenit(@nev);  
SELECT @nev AS `Név` FROM dual;
```

Áron

Aron